Overview
Welcome to Semantica

Core Concepts
Core Concepts
Modules
Choose the Right Module
Glossary

Guides
Architecture
Learning More
GraphRAG: Graph-Augmented Retrieval
Ingest Anything
LLM Integrations
Pipeline Builder
Multi-Agent Systems
Decision Intelligence
Context Graphs
Agent Memory
Provenance & Audit Trails
Semantic Extraction
Ontology Management
Reasoning & Rules
Export & Serialization
Deduplication & Entity Merging
MCP Server
SHACL Validation
Conflict Detection & Resolution
Change Management & Versioning
Policy Engine
Visualization
Distance Intelligence
Graph Analytics

Integrations
Agno Integration
CrewAI Integration
LangChain Integration
Docling Integration
Snowflake Integration
Databricks Integration
Salesforce Integration

Graph Stores
Apache AGE Graph Store

Vector Stores
pgvector Store

FAQ
FAQ

Community
Community
Community Projects
Contributing
Governance
Citation
License

Guides

Decision Intelligence

How Semantica records, stores, traces, and queries AI agent decisions as first-class knowledge graph objects — with causal chains, precedent search, policy enforcement, and full explainability.

`AgentContext.record_decision()` stores every AI decision as a node in the knowledge graph, linked by causal edges to the decisions that preceded it and the outcomes that followed. Use it to build an auditable reasoning trail — one that lets you reconstruct, six months later, exactly which classification caused which escalation, and which policy was checked before it was recorded.

On this page
What Is Decision Intelligence?
Why Use Decision Intelligence?
When To Use / When Not To Use
API Architecture Overview
Recording the First Decision
Searching Precedents Before Deciding
Building a Causal Chain
Generating an Explainability Report
Gating Decisions Against Policy
Generating a Decision Audit Report
Domain Examples
Persisting Decisions Across Restarts
Common Pitfalls
Related Guides

What Is Decision Intelligence?

Decision Intelligence records and analyzes an agent's own decisions as structured data that can be queried, analyzed, and reused. Instead of decisions disappearing after execution, they become persistent graph nodes with searchable metadata, reasoning chains, and causal relationships.

Decision Intelligence records decisions by capturing the scenario, reasoning, outcome, confidence, and decision maker for each choice the agent makes. These decisions become queryable nodes in your knowledge graph.

Decisions become graph nodes that can be linked causally (Decision A caused Decision B), searched by similarity (find decisions like this scenario), and analyzed statistically (confidence trends, common outcomes).

The goal is auditability, explainability, precedent search, and causal tracing. You can trace why decisions were made, find similar past decisions for consistency, and understand the full causal chain from initial detection to final action.

Decision Intelligence vs. Agent Memory: Agent Memory stores external knowledge (documents, facts, observations). Decision Intelligence stores internal decisions (classifications, approvals, actions the agent itself made).

Decision Intelligence vs. Reasoning: Reasoning derives new facts from existing data using logical rules. Decision Intelligence records the choices and judgments the agent made during problem-solving.

Decision Intelligence vs. Graph Analytics: Graph Analytics analyzes the structural properties of your knowledge graph. Decision Intelligence focuses specifically on the decision-making process and its audit trail.

Why Use Decision Intelligence?

Auditable AI actions. Every decision is recorded with reasoning, confidence, and timestamp, creating a complete audit trail for AI behavior in production systems.

Explainability. When stakeholders ask "why did the system do X?", you can trace the exact decision chain that led to that action, including intermediate reasoning steps.

Precedent reuse. Before making new decisions, agents can search for similar past scenarios and their outcomes, promoting consistency and learning from previous experience.

Causal analysis. Understand how early decisions cascade into later outcomes by following causal relationships between linked decision nodes.

Governance and compliance. Policy engines can gate decisions against compliance rules, and all policy applications are recorded for regulatory audit.

When To Use / When Not To Use

Use Decision Intelligence when:

- Building autonomous agents that make consequential choices
- Implementing decision workflows requiring audit trails
- Operating under compliance requirements (financial services, healthcare, defense)
- Building approval systems with multiple decision points
- Working in risk-sensitive environments where decisions must be explainable

Do not use when:

- Building stateless chatbots that only retrieve information
- Implementing simple RAG systems without decision-making
- Creating read-only information retrieval applications
- Building applications that never make actionable decisions requiring audit trails

API Architecture Overview

Decision Intelligence coordinates three main components:

AgentContext serves as the high-level orchestration layer. It provides `record_decision()`, `find_precedents()`, and causal chain methods while managing the underlying storage and retrieval systems.

PolicyEngine handles policy evaluation and compliance checking. It stores policy rules as graph nodes and validates decisions against those rules before they're recorded.

DecisionRecorder specializes in recording structured decision data, managing approval chains, and handling policy exceptions when decisions need to bypass normal rules.

ⓘ Decision tracking requires both a `VectorStore` (for embedding-based precedent search) and a `ContextGraph` (for causal graph storage). Set `decision_tracking=True` on `AgentContext` — omitting `ContextGraph` raises a `RuntimeError` at call time. `VectorStore` is required by `AgentContext` itself: leaving the argument out raises a `TypeError` from Python's argument binding, while passing `vector_store=None` raises a `ValueError` during initialization.

Recording the First Decision

The most common entry point is `AgentContext.record_decision()`. It writes a `Decision` node into the graph, generates embeddings for hybrid similarity search, and returns a UUID you use to link subsequent decisions causally.

```python
from semantica.context import AgentContext, ContextGraph
from semantica.vector_store import VectorStore

graph   = ContextGraph(advanced_analytics=True)
context = AgentContext(
    vector_store=VectorStore(backend="faiss", dimension=768),
    knowledge_graph=graph,
    decision_tracking=True,
)

# The system has just classified a new threat cluster.
# Record the classification decision before taking any action.
classification_id = context.record_decision(
    category       = "threat_classification",
    scenario       = "Unattributed C2 cluster using HAMMERTOSS-like Twitter dea
    reasoning      = "Infrastructure overlaps known APT29 hosting ASN; TTP T110
    outcome        = "classified_as_apt29_cluster",
    confidence     = 0.88,
    decision_maker = "cti_pipeline_v2",
    entities       = ["apt29", "hammertoss", "twitter_c2"],
)

print("Decision recorded:", classification_id)
# → "Decision recorded: dec_a3f2b1c4-..."
```

The `decision_maker` field identifies the component, workflow, agent, or system that produced this decision. Use consistent identifiers like `"cti_pipeline_v2"`, `"analyst_chen"`, or `"risk_model_v3"` to enable filtering and analysis by decision source.

The `Decision` dataclass that backs this node has the following fields — these are what get stored and searched:

```python
from semantica.context import Decision
from datetime import datetime

d = Decision(
    decision_id    = None,                  # required arg — None/"" auto-generat
    category       = "threat_classification",
    scenario       = "Unattributed C2 cluster",
    reasoning      = "Infrastructure overlaps APT29 ASN",
    outcome        = "classified_as_apt29_cluster",
    confidence     = 0.88,                 # float 0.0–1.0
    timestamp      = datetime.now(),
    decision_maker = "cti_pipeline_v2",
    # optional fields:
    valid_from     = "2025-07-01T00:00:00",    # ISO datetime
    valid_until    = "2025-09-30T23:59:59",    # ISO datetime
    metadata       = {"source_feed": "isac_partner_b"},
)
```

To actually store a decision built this way, pass its fields to `ContextGraph.add_decision()` as keyword arguments — this is the alternative to `record_decision()` for cases where you want `valid_from` / `valid_until` or extra metadata fields alongside the required ones:

```python
decision_id = graph.add_decision(
    category       = "threat_classification",
    scenario       = "Unattributed C2 cluster",
    reasoning      = "Infrastructure overlaps APT29 ASN",
    outcome        = "classified_as_apt29_cluster",
    confidence     = 0.88,                 # float 0.0–1.0
    decision_maker = "cti_pipeline_v2",
    # optional fields:
    valid_from     = "2025-07-01T00:00:00",    # ISO datetime
    valid_until    = "2025-09-30T23:59:59",    # ISO datetime
    source_feed    = "isac_partner_b",         # extra kwargs are stored as meta
)
```

⚠ Only pass keyword arguments to `add_decision()`, not a pre-built `Decision` object. `add_decision(Decision(...))` stores the node directly and skips the indexing step that `record_decision()` performs, so the decision becomes invisible to `find_precedents()`, `get_causal_chain()`, and `get_decision_insights()`, and `trace_decision_causality()` raises `ValueError` if you call it on one. The keyword-argument form above does not have this problem — it delegates to `record_decision()` internally. Note that, like `record_decision()`, it always generates its own `decision_id` (returned from the call); there is no way to force a specific ID.

Searching Precedents Before Deciding

Before making a significant call, the system should search past decisions for similar scenarios. This is how you prevent the same cluster being classified differently across two agent runs — the second agent finds the first agent's decision and uses it as a prior.

```python
# Search before classifying a new unattributed cluster
precedents = context.find_precedents(
    "unattributed C2 cluster Twitter dead-drop infrastructure",
    limit=5,
)

for p in precedents:
    print("[{:.2f} confidence] {} → {}".format(p.confidence, p.category, p.outc
    print("  Reasoning: {}".format(p.reasoning[:80]))
    print("  Similarity: {:.3f}".format(p.metadata.get("similarity_score", 0)))
```

Hybrid search blends two signals: lexical overlap between the query and each decision's `scenario`, `reasoning`, and `entities` text (weight 0.7 — word-level Jaccard similarity, with a character-bigram fallback for CJK-style queries), and structural similarity based on how many other nodes each decision connects to in the graph (weight 0.3, only computed when the graph was built with `advanced_analytics=True`). The result is a ranked list of `Decision` objects, filtered to those scoring at least `similarity_threshold` (default 0.5) — because the match is lexical rather than embedding-based, precedents phrased very differently from the query may not surface even if they describe a similar scenario.

Building a Causal Chain

Decisions rarely exist in isolation. A classification decision causes an escalation decision, which causes a containment decision. Linking them with causal edges lets you traverse the chain in either direction — upstream to understand what caused an outcome, downstream to see what an early decision triggered.

```python
# The classification above caused an escalation
escalation_id = context.record_decision(
    category       = "escalation",
    scenario       = "APT29 cluster confirmed — active C2 beaconing to NATO con
    reasoning      = "Classification confidence 0.88 exceeds 0.80 escalation th
    outcome        = "escalated_to_soc_tier2",
    confidence     = 0.95,
    decision_maker = "escalation_engine",
)

# Link the two decisions: classification caused the escalation
graph.add_causal_relationship(classification_id, escalation_id, "CAUSED")

# The escalation influenced a patch prioritization decision
patch_id = context.record_decision(
    category       = "patch_priority",
    scenario       = "CVE-2024-3400 present in two NATO contractor VPN applianc
    reasoning      = "Active exploitation by classified APT29 cluster elevates
    outcome        = "prioritized_cve_2024_3400_p0",
    confidence     = 0.97,
    decision_maker = "patch_engine",
)

graph.add_causal_relationship(escalation_id, patch_id, "INFLUENCED")
```

Now trace the chain from the patch decision back to its root cause:

```python
upstream = context.get_causal_chain(
    patch_id,
    direction = "upstream",
    max_depth = 5,
)

print("Causal chain upstream from patch prioritization:")
for d in upstream:
    depth = d.metadata.get("causal_distance", "?")
    print("  [depth {}] {} → {}  (confidence={:.2f})".format(
        depth, d.category, d.outcome, d.confidence
    ))
# [depth 1] escalation → escalated_to_soc_tier2  (confidence=0.95)
# [depth 2] threat_classification → classified_as_apt29_cluster  (confidence=0.
```

And trace downstream from the original classification to see everything it triggered:

```python
downstream = context.get_causal_chain(
    classification_id,
    direction = "downstream",
    max_depth = 5,
)
print("Downstream decisions triggered:", len(downstream))
for d in downstream:
    print("  → {} [{}]".format(d.outcome, d.category))
```

Generating an Explainability Report

`trace_decision_explainability` gives you the full picture in one call: upstream causes, downstream effects, and total connection count. This is what you attach to a post-mortem or audit report.

```python
explanation = context.trace_decision_explainability(patch_id)

print("Decision:", patch_id)
print("Total graph connections :", explanation["total_connections"])
print("Upstream causes         :", len(explanation.get("upstream_decisions", []
print("Downstream effects      :", len(explanation.get("downstream_decisions",
```

For deeper causal analysis with confidence decay and distance bands, use `trace_decision_causality` on the graph directly:

```python
chains = graph.trace_decision_causality(patch_id, max_depth=5)

for chain in chains:
    print("Chain: {} hops | band={} | decay={:.3f}".format(
        chain["hop_count"], chain["distance_band"], chain["confidence_decay"]
    ))
    print("  Interpretation:", chain["interpretation"])
    # e.g. "Decision chain spans 2 hops in the 'near' band with 84% confidence
    #        — causal attribution is reliable."
```

Gating Decisions Against Policy

Before recording a high-stakes decision, check it against a versioned policy. The `PolicyEngine` stores `Policy` nodes in the graph and gates `Decision` objects against their rules.

```python
from semantica.context import PolicyEngine, Policy, Decision
from datetime import datetime

engine = PolicyEngine(graph_store=graph)

engine.add_policy(Policy(
    policy_id   = "cti_confidence_gate",
    name        = "CTI Minimum Confidence Policy",
    description = "All threat classifications must have confidence >= 0.80",
    rules       = {"min_confidence": 0.80, "requires_reasoning": True},
    category    = "threat_classification",
    version     = "1.0",
    created_at  = datetime.now(),
    updated_at  = datetime.now(),
))

d = Decision(
    decision_id    = "dec_low_conf",
    category       = "threat_classification",
    scenario       = "Possible APT29 activity — weak signals only",
    reasoning      = "Single IP overlap, no TTP match",
    outcome        = "classified_as_apt29_tentative",
    confidence     = 0.62,   # below the 0.80 threshold
    timestamp      = datetime.now(),
    decision_maker = "cti_pipeline_v2",
)

if engine.check_compliance(d, "cti_confidence_gate"):
    # Pass fields as kwargs, not the Decision object itself — see the
    # warning above. add_decision() generates its own decision_id.
    decision_id = graph.add_decision(
        category=d.category, scenario=d.scenario, reasoning=d.reasoning,
        outcome=d.outcome, confidence=d.confidence, decision_maker=d.decision_m
    )
    engine.record_policy_application(decision_id, "cti_confidence_gate", "1.0")
    print("Decision recorded — policy compliant.")
else:
    print("Decision blocked — confidence 0.62 below policy minimum 0.80.")
    # → "Decision blocked — confidence 0.62 below policy minimum 0.80."
```

When a high-urgency situation requires bypassing the policy gate, record the exception with the approver identity and justification:

```python
from semantica.context import DecisionRecorder

recorder = DecisionRecorder(graph_store=graph)

exception_id = recorder.record_exception(
    decision_id     = "dec_low_conf",
    policy_id       = "cti_confidence_gate",
    reason          = "Active exploitation in progress — cannot wait for higher
    approver        = "ciso_director",
    approval_method = "slack_dm",
    justification   = "Time-critical incident response; manual CISO sign-off ob
)
print("Policy exception recorded:", exception_id)
```

For multi-level approval workflows, use `DecisionRecorder.record_approval_chain()` with a graph database backend (for example Neo4j/FalkorDB). The in-memory `ContextGraph` examples used in this guide do not support approval-chain persistence via `execute_query()`.

Generating a Decision Audit Report

At the end of a shift or incident, `get_decision_insights` produces a statistical summary of every decision in the graph — useful for shift handover notes and compliance reporting.

```python
insights = graph.get_decision_insights()

print("Total decisions today :", insights["total_decisions"])
print("Confidence — mean={:.2f}  min={:.2f}  max={:.2f}".format(
    insights["confidence_stats"]["mean"],
    insights["confidence_stats"]["min"],
    insights["confidence_stats"]["max"],
))
print("\nDecisions by category:")
for cat, count in sorted(insights["categories"].items(), key=lambda x: -x[1]):
    print("  {:35s} {}".format(cat, count))
print("\nOutcomes:")
for outcome, count in sorted(insights["outcomes"].items(), key=lambda x: -x[1])
    print("  {:35s} {}".format(outcome, count))
```

Sample output:

```
Total decisions today : 47
Confidence — mean=0.87  min=0.62  max=0.99

Decisions by category:
  threat_classification               18
  patch_priority                      12
  escalation                           9
  containment                          8

Outcomes:
  classified_as_apt29_cluster         11
  prioritized_p0_patch                12
  escalated_to_soc_tier2               9
  isolated_host                        8
```

Domain Examples

Defense — CTI/Threat **Security — SOC/Incident** **Life Science — Clinical/Pharma** **Banking — Ri**

A clinical AI assistant records treatment modification decisions with guideline precedents, links them causally to the diagnostic decisions that preceded them, and generates a structured decision record for MDT review and regulatory audit.

```python
from semantica.context import AgentContext, ContextGraph, PolicyEng:
from semantica.context import Decision, DecisionRecorder
from semantica.vector_store import VectorStore
from datetime import datetime

graph   = ContextGraph()
engine  = PolicyEngine(graph_store=graph)
context = AgentContext(
    vector_store=VectorStore(backend="faiss", dimension=768),
    knowledge_graph=graph,
    decision_tracking=True,
)

engine.add_policy(Policy(
    policy_id   = "clinical_confidence_gate",
    name        = "Clinical Decision Confidence Gate",
    description = "Treatment decisions require confidence >= 0.90",
    rules       = {"min_confidence": 0.90},
    category    = "treatment_modification",
    version     = "1.0",
    created_at  = datetime.now(),
    updated_at  = datetime.now(),
))

# Diagnostic decision precedes treatment decision
diag_id = context.record_decision(
    category       = "diagnosis_assessment",
    scenario       = "Patient eGFR 28 mL/min/1.73m2, CKD Stage 4, current metfo
    reasoning      = "eGFR 28 confirms CKD Stage 4; below 30 threshold for metf
    outcome        = "confirmed_ckd_stage4_metformin_contraindicated",
    confidence     = 0.99,
    decision_maker = "clinical_ai_v3",
)

# Treatment modification caused by diagnostic assessment
treat_id = context.record_decision(
    category       = "treatment_modification",
    scenario       = "Metformin discontinuation required — eGFR 28 below contra
    reasoning      = "NICE NG28 and BNF both contraindicate metformin at eGFR <
    outcome        = "discontinue_metformin_initiate_gliclazide",
    confidence     = 0.97,
    decision_maker = "clinical_ai_v3",
)
graph.add_causal_relationship(diag_id, treat_id, "CAUSED")

# MDT audit report
chain = context.get_causal_chain(treat_id, direction="upstream", max_depth=3)
print("MDT Decision Audit — Treatment Modification")
print("=" * 50)
for d in chain:
    print("Caused by: [{}] {} (confidence={:.0%})".format(
        d.category, d.outcome, d.confidence
    ))

insights = graph.get_decision_insights()
cs = insights["confidence_stats"]
print("\nSession decisions: {}  |  mean confidence: {:.2f}".format(
    insights["total_decisions"], cs["mean"]
))
```

Persisting Decisions Across Restarts

When using the local `ContextGraph`, save at the end of every session and load at the start of the next. All decision nodes, causal edges, and FAISS embeddings are restored.

```python
# End of session
context.save("agent_state/")
# Writes: agent_state/knowledge_graph.json + FAISS index

# Start of next session
context = AgentContext(
    vector_store=VectorStore(backend="faiss", dimension=768),
    knowledge_graph=ContextGraph(),
    decision_tracking=True,
)
context.load("agent_state/")

# All past decisions are searchable immediately
results = context.find_precedents("APT29 infrastructure attribution", limit=5)
```

Common Pitfalls

Recording decisions without linking causal relationships. Isolated decision nodes provide less insight than connected decision chains. Use `add_causal_relationship()` to link related decisions and enable causal tracing.

Creating isolated decision nodes. Decisions gain value when connected to entities, other decisions, or outcomes in your graph. Link decisions to relevant entities using the `entities` parameter.

Recording too many low-value decisions. Not every minor choice needs permanent recording. Focus on consequential decisions that affect outcomes, require audit trails, or benefit from precedent search.

Treating precedent similarity as proof. High similarity scores indicate related scenarios, not identical situations. Use precedents as guidance while considering the specific context of each new decision.

Using Decision Intelligence when simple retrieval is sufficient. If your system only retrieves information without making actionable choices, traditional search or Agent Memory may be more appropriate than decision tracking.

Related Guides

- **Context Graphs** — how `ContextGraph` stores decision nodes and causal edges
- **Distance Intelligence** — `trace_decision_causality()` annotates causal chains with confidence decay and distance bands
- **Provenance** — W3C PROV-O audit trail that wraps decision records in standards-compliant provenance
- **MCP Server** — expose decision recording and precedent search to LLM agents via the `record_decision` and `find_precedents` tools
- **Change Management** — checkpoint decision state with `flush_checkpoint()` for versioned snapshots

‹ Multi-Agent Systems

Context Graphs ›

Powered by mintlify